Journal of Real-Time Image Processing (2025) 22:139
https://doi.org/10.1007/s11554-025-01720-z

RESEARCH

FAN-YOLO: Real-time driver behavior detection based on full-layer aggregation network of YOLO

Xinsu Gan[1] · Lidong Huang[1] · Yan Yuan[1] · Yan Deng[1]

Received: 24 February 2025 / Accepted: 14 June 2025 / Published online: 1 July 2025
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2025

Abstract

Driver behavior detection is crucial to improving traffic safety and protecting driver health. Traditional driver behavior monitoring methods often rely on complex models with high computing resources, which are difficult to meet the requirements of real-time and lightweight. This study proposes a full-layer aggregation network (FAN) based on the YOLO (You Only Look Once) architecture, and constructs a lightweight driver behavior detection algorithm called FAN-YOLO. FAN-YOLO combines the YOLO object detection framework and lightweight network structure. Based on the standard YOLO framework, it changes the network structure of the neck layer path aggregation network (PAN) and adopts FAN to alleviate the problems of complex PAN path interactions and information redundancy. At the same time, a new feature fusion module is proposed in the backbone network, which we call RC2A, to extract better feature information. Then, the spatial pyramid pooling (SPP) network is optimized to obtain SPPF_WD, so that FAN-YOLO can detect driver behavior more accurately. FAN-YOLO is experimented on two datasets of abnormal driver behavior. The experimental results show that the number of parameters of FAN-YOLO is only 2.2M, which is 18.5% lower than the SOTA model YOLOv8-N in YOLO. The calculation amount is 6.2G, which is 10% lower. On the improved datasets State Farm v2 and YawDD v2, AP_{50}^{val} can reach 86.8% and 97.5%, 1.3% higher than YOLOv8-N. The response speed is 1.9ms, which is 29.6% higher.

Keywords Driver behavior detection · FAN · Lightweight network · YOLO

1 Introduction

According to the World Health Organization (WHO) Global Plan for the Decade of Action for Road Safety [1], road traffic accidents cause nearly 1.3 million deaths and 50 million injuries each year worldwide, of which nearly one-fifth are caused by driver distraction [2]. However, the key to solving the problem of distracted driving is how to correctly identify the driver in an abnormal driving state and then provide early warning. Therefore, correctly identifying driver distraction is crucial.

In order to correctly identify driver distraction behaviors, computer vision-based distraction detection systems become crucial. Initially, some studies [3, 4] focused on mobile phone detection, using support vector machines (SVM) to check the hand position for the presence of mobile phone use. But in real life, mobile phones are not the only cause of accidents, so researchers began to pay attention to a wider range of distraction behaviors. Various abnormal behaviors of drivers in the car were included in the research objects. After 2017, more researchers [5–10] used convolutional neural network (CNN) architecture to classify the driver's posture, including making phone calls, adjusting equipment, turning the head to observe, eating and drinking, dozing off and other distracting behaviors. Shen J team [11]used a lightweight CNN to read the instrument indication. This study contributed to preventing drivers from speeding. These studies not only observed the state of the driver's hands, but also paid attention to the driver's head, face and in-car equipment.

✉ Lidong Huang
hldwinter@163.com

Xinsu Gan
xxgan1201@163.com

Yan Yuan
yuanyan_2022@163.com

Yan Deng
dengy1207@163.com

[1] School of Mathematics and Computer Science, Yunnan Minzu University, Kunming 650504, Yunnan, China

However, the performance of CNN-based models depends on changes in lighting, camera angles, and occlusions, i.e., underestimation and missed detection. On the other hand, the computational difficulty level of CNN networks requires a lot of processing power, which can sometimes be a limiting factor in real-time use. YOLO is a fast and efficient network structure that can further alleviate the performance inefficiency of CNN. In 2021, YS Poon et al. [12] conducted a comparison using multiple YOLO variants, such as YOLOv3-tiny, YOLOv3-tiny-3 l, YOLO-fastest, and YOLO-fastest-xl, to evaluate their performance in driver distracted behavior detection. Subsequently, BG Amira and MM Zoulikha [13] proposed a tracking system for driver fatigue detection that combines YOLO, Haar feature cascade classifiers, and Elman recursive neural networks. YOLO is used to detect the driver's facial area, and Haar cascade classifiers are used to identify key parts such as eyes and mouths. The time series changes of these features are analyzed through Elman recurrent neural network (ERNN) to determine the driver's fatigue state. In 2022, Qin L et al. [14] constructed an enhanced eye movement target detection dataset and designed the ID-YOLO architecture to analyze the driver's eye movement data to simulate the driver's visual attention. In 2024, K Zhou et al. [15] added attention-based intra-scale feature interaction (AIFI) and triple feature encoding (TFE) modules on the basis of YOLOv5 to improve the detection accuracy of driver distraction behavior. However, the introduction of AIFI and TFE increased the complexity of the network and limited the deployment of the algorithm. Debsi A et al. [16] mentioned that an important application of the YOLO algorithm is to identify behavioral patterns that distract signals. However, in complex light occlusion environments, the improvement of processing speed becomes a major issue. Therefore, he proposed the ME-YOLOv8 model, which added multi-head self-attention (MHSA) and efficient channel attention (ECA) on the basis of YOLOv8, and constructed a dedicated fatigue detection dataset (DDFDD), which performed well in indicators such as accuracy, processing speed, and recall.

Although the YOLO architecture can solve some of the computational complexity problems of CNN, the above studies still cannot meet the real-time requirements, and they ignore the importance of seat belts to drivers and passengers. A perfect driver behavior detection system should include seat belt detection. This paper first collects and produces a data set for driver behavior detection. In order to build a system that can meet the real-time requirements, we propose FAN-YOLO. In the traditional YOLO architecture, the feature pyramid network (FPN) [17] adopts a top-down unidirectional information flow transmission, and the PAN [18] inherits the top-down structure of FPN, adds bottom-up path enhancement, and writes it as a bidirectional information flow. However, FPN and PAN have complex path interactions and information redundancy. In FAN-YOLO, we propose FAN to replace PAN to solve the above problems. We propose FAN to replace PAN in FAN-YOLO to solve these problems. In addition, the bottleneck module and the SPP [19] network in the YOLO backbone network are optimized, and the contextual self-attention mechanism is added to the residual structure to form a new bottleneck module ResCoTBlock and feature fusion module RC2A, which effectively reduces the parameters and the calculation amount while improving the detection accuracy of the original YOLO series model (YOLOv5N-YOLOv10N) for driver distraction.

The main contributions of this paper are as follows:

1. Propose the FAN architecture and use this structure to replace the PAN structure commonly used in the YOLO series. While reducing the parameters and calculation amount, it also solves the shortcomings of the large number of paths, indirect interaction, and information redundancy in the PAN network.
2. Introduce the contextual self-attention mechanism, design a new bottleneck structure ResCoTBlock and feature fusion module RC2A, and introduce RC2A into the YOLO backbone network as the backbone network of the FAN-YOLO model.
3. Re-examine the traditional SPP architecture and design a pooling network named SPPF_WD, which uses an incremental pooling operation with a pooling operation core of 5 to obtain a larger receptive field and enable the model to more accurately detect driver distraction behavior.
4. A large number of experiments on datasets show that the method in this paper effectively reduces the computational complexity while improving the accuracy of driver behavior detection. In addition, compared with other lightweight architectures, the method in this paper achieves significantly better results in all aspects.

The remainder of this paper is organized as follows: Section 2 reviews related work. Section 3 elaborates on the proposed model. Section 4 presents the experiments and results analysis. Finally, Sect. 5 summarizes and prospects this paper.

2 Related work

In this section, we describe the lightweight YOLO architecture. After years of development, the YOLO series of models has become one of the representatives in the field of real-time object detection. YOLOv1 to YOLOv3 [20–22] laid the foundation of YOLO, established a single-stage detection framework, and adopted a structure

consisting of a backbone network, a neck, and a detection head. YOLOv4 [23] and YOLOv5 [24] optimized the previously used Darknet53 backbone network, introduced FPN and PAN, proposed improved data augmentation strategies and more types of model variants. YOLOv6 [25] and YOLOv7 [26] introduced the reparameterization method into the YOLO series of models and proposed the E-ELAN structure to enhance the model capabilities without destroying the original gradient path. YOLOv8 [27] has matured after the previous preparation. Through a deeper network and enhanced training strategy, it has improved its adaptability to complex scenes and large-scale datasets, and improved the performance of small object detection and multi-target scenes. YOLOv9 [28] proposed programmable gradient information with auxiliary branches and an efficient generalized ELAN strategy. YOLOv10 [29] focused on improving the detection head, adopting a dual-head architecture, using one-to-many allocation during training and one-to-one allocation during inference, and achieved efficient detection without NMS. Among the many YOLO versions, YOLOv8 has become the SOTA model in the YOLO series with its excellent detection accuracy and stable model structure.

Since YOLOv5 introduced FPN and PAN as the neck layer network of the model, subsequent versions have regarded them as essential components of YOLO. However, PAN has the disadvantages of complex path interaction and information redundancy. As shown in Fig. 1, the features of each input detection head carry the features of large, medium and small targets. The original intention of designing three detection heads is that the low-level detection head detects small targets, the middle-level detection head detects medium targets, and the high-level detection head detects large targets. However, there are many contradictions between small targets and medium targets, and between medium targets and large targets. Generally speaking, there is no contradiction between small targets and large targets because their sizes are easy to distinguish. Therefore, this paper designs FAN, so that the high-level detection head only carries the features of large targets and medium targets, the low-level detection head carries the features of small targets and medium targets, and the middle-level detection head serves as a supplement to them. This design reduces the information redundancy of PAN.

3 Method

In order to solve the shortcomings of PAN in the traditional YOLO structure, such as slow speed, complex cross-layer information exchange and information redundancy, this paper designs FAN-YOLO as shown in Fig. 2.

The main improvements of FAN-YOLO include the designed FAN, the proposed feature fusion module RC2A, and the improved pyramid pooling network SFFP_WD. In the backbone network module, this paper adopts the backbone network structure of YOLO, and designs a new bottleneck module on its basis to make it efficient in detection while keeping its backbone network lightweight. Then, the SFFP_WD network is added to the deepest level to generate feature information B1, B2, B3, and B4. These four features are features of different levels in the backbone network, among which B1 has only one low-level feature, B2 and B3 contain features of 3 different levels respectively, and B4 generated by SPPF_WD is a high-level feature that carries a lot of semantic information. Finally, the feature information input to Predict-1 only contains large and medium targets, the feature information input to Predict-3 only contains small and medium targets, and the feature input to Predict-2 carries the most contradictory feature information of the medium target as an important supplement.

3.1 Structure of FAN

FAN is designed as the neck layer network of this paper. It is proposed to solve the problems of complex cross-layer information exchange and information redundancy of FPN and PAN, so that each detection head can work relatively specifically on features of different scales. FAN first introduces an efficient path design, and adjusts feature consistency through the feature alignment module Align_3In to reduce the propagation of redundant information. The Align_3In module is shown in Fig. 3. Its input is provided

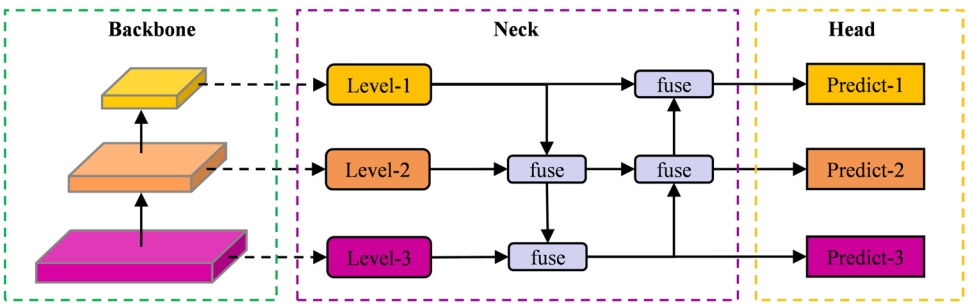


Fig. 1 Structure of YOLO

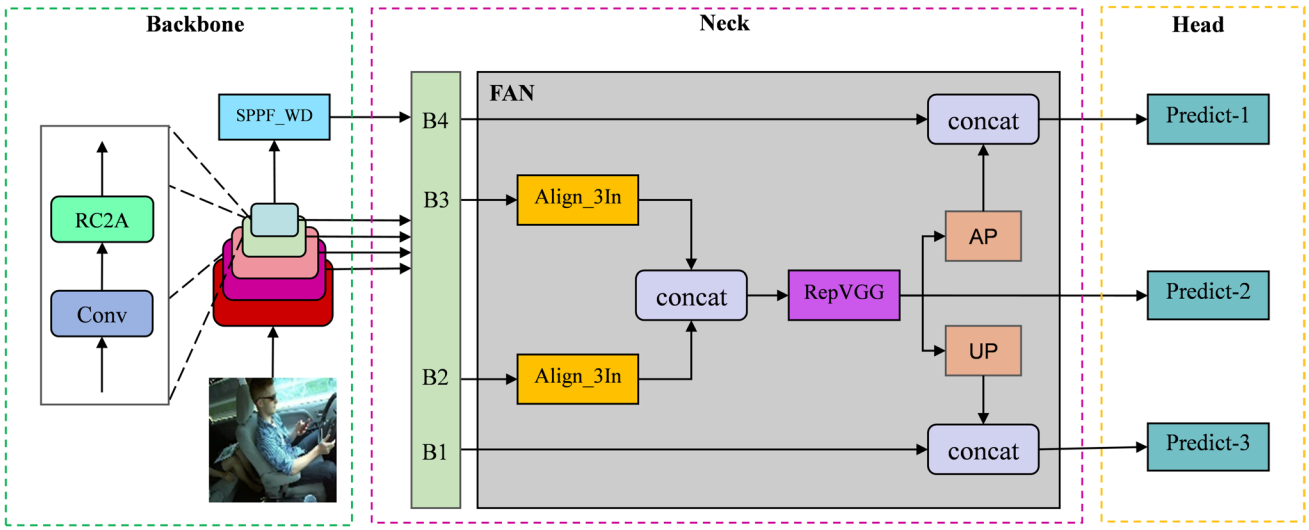


Fig. 2 Specific structure of FAN-YOLO

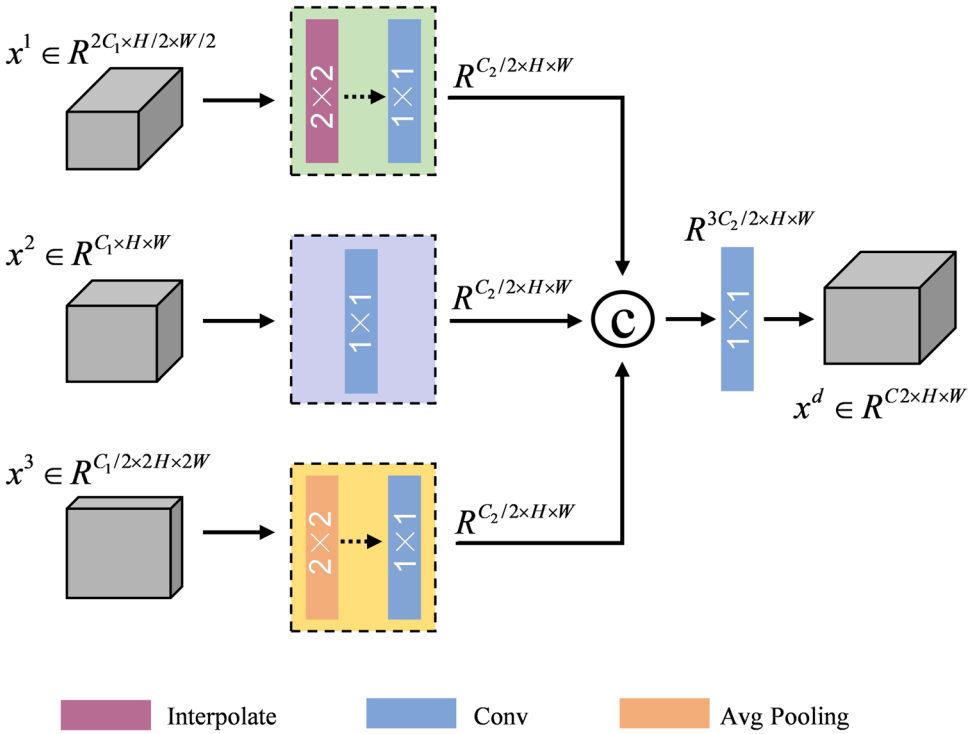


Fig. 3 Align_3In structure diagram. c represents channel splicing

by feature maps of three different levels. The low-level feature maps enhance semantic information through the auxiliary method of pooling operation, and the high-level feature maps alleviate the loss of position information through interpolation method. Subsequently, the low-level feature maps with strong semantic information and the high-level feature maps with strong position information are concatenated with the middle layer according to the channel to obtain a more comprehensive feature information. The concatenated feature maps are combined with the RepVGG module. During training, the diversity of the receptive field is increased by using a multi-branch model, and during inference, the multi-branch model is decoupled into an inference smoothing model through structural reparameterization. RepVGG uses its structured lightweight convolution design to reduce the inference speed of the model and reduce memory utilization. Finally, the obtained rich feature information is connected with the corresponding level, so that the feature map

of the corresponding level can contain the feature information of the corresponding task more specifically.

3.1.1 RepVGG module

RepVGG [30] is a simple architecture that has a good speed–accuracy trade-off compared to state-of-the-art ConvNets. RepVGG follows the residual edge of ResNet, its structure is shown in Fig. 4. Specifically, RepVGG uses 3×3 convolution, 1×1 convolution, and entity branches to construct the model during training. This training method avoids the problem of gradient vanishing, but it slows down the efficiency of inference. Therefore, during model inference, the training multi-branch model is decoupled into an inference smooth model through structural reparameterization. Figure b shows the model structure used during training, and Figure c shows the model structure used during inference, where a BN layer is added before the output of each branch in the training model. The use of structural reparameterization separates the planar structure of inference time from the multi-branch topology of training time, which results in a significant acceleration during inference.

Structural reparameterization improves the efficiency and performance of the model by merging multiple computational modules into one during the inference phase. In one RepConv, there are multiple equivalent replacement operations, the merger of convolution and BN layer, the 3×3 convolution kernel replaces the 1×1 convolution kernel, and the 3×3 convolution replaces the identity, as shown in Fig. 5. Although there are many different components during training, so that the training time information flow of the building block is $y = x + g(x) + f(x)$, after reparameterization

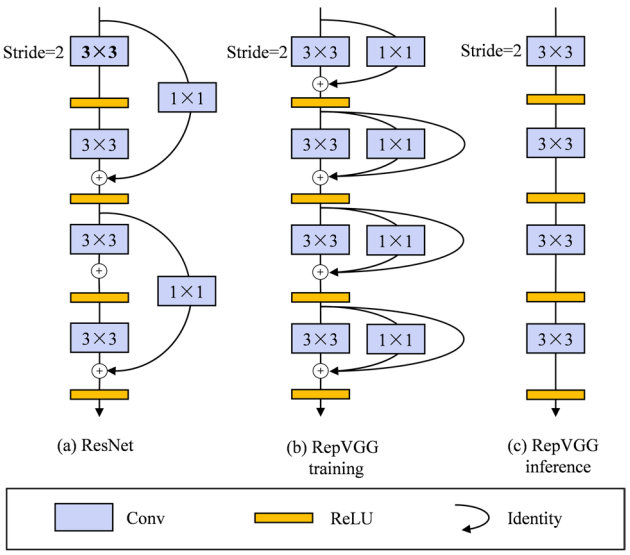


Fig. 4 RepVGG structure diagram

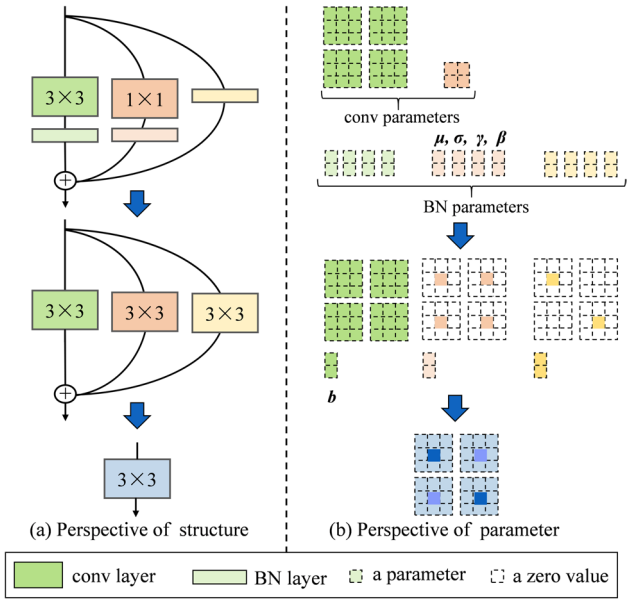


Fig. 5 Structural reparameterization of RepVGG module

technology, there is only 3×3 convolution in RepVGG, which greatly improves the model performance.

RepVGG extensively uses Conv+BN layers and merges layers to reduce the number of layers. The inference process is as follows:

Conv layer:

$$Conv(x) = W(x) + b \tag{1}$$

BN layer:

$$BN(x) = \gamma * \frac{(x - mean)}{\sqrt{\text{var}}} \tag{2}$$

bring the Conv layer structure into the BN formula:

$$BN(Conv(x)) = \gamma * \frac{W(x) + b - mean}{\sqrt{\text{var}}} + \beta \tag{3}$$

further simplification:

$$BN(Conv(x)) = \frac{\gamma * W(x)}{\sqrt{\text{var}}} + (\frac{\gamma * (b - mean)}{\sqrt{\text{var}}} + \beta) \tag{4}$$

Where $\frac{\gamma * W(x)}{\sqrt{\text{var}}}$ is the combined weight and $\frac{\gamma * (b - mean)}{\sqrt{\text{var}}} + \beta$ is the combined offset.

3.2 Feature fusion module RC2A

The design of the feature fusion module RC2A is to increase the richness of features. Its design is inspired by the c2f module of YOLOv8, but it combines the design of the Ghost

module and ResCoTBlock. The Ghost module reduces the computation of redundant features through lightweight convolution operations, which can effectively improve the computational efficiency of the network. ResCoTBlock is the bottleneck structure designed in this paper, which is used to replace the Bottleneck module in c2f. The splitting and splicing method is adopted to make the output feature information richer. With the help of the residual structure, it mainly solves the problem of insufficient traditional convolution feature extraction. The structure diagram of RC2A is shown in Fig. 6.

3.2.1 Ghost module

The Ghost module was proposed in GhostNet [31]. Considering the widespread redundancy in the intermediate feature maps calculated by ConvNets, it is necessary to reduce unnecessary resources, that is, to reduce the convolution filters that generate them. Therefore, the design idea of the Ghost module is to first generate a set of feature maps through ordinary convolution, and then generate more feature maps through simple linear transformation. Specifically, given the input data X, ordinary convolution generates m feature maps Y', and then generates n feature maps Y through simple linear transformation ϕ, where n >m. Without changing the input and output feature maps, the amount of calculation and parameters of the model is greatly reduced. The Ghost module is shown in Fig. 7.

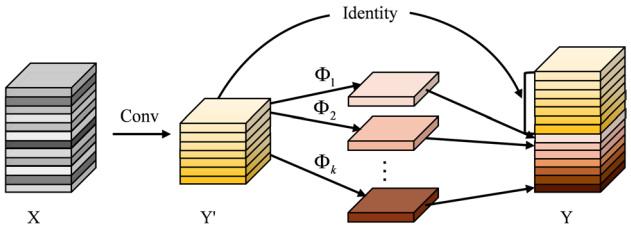


Fig. 7 Ghost module

3.2.2 ResCoTBlock bottleneck module

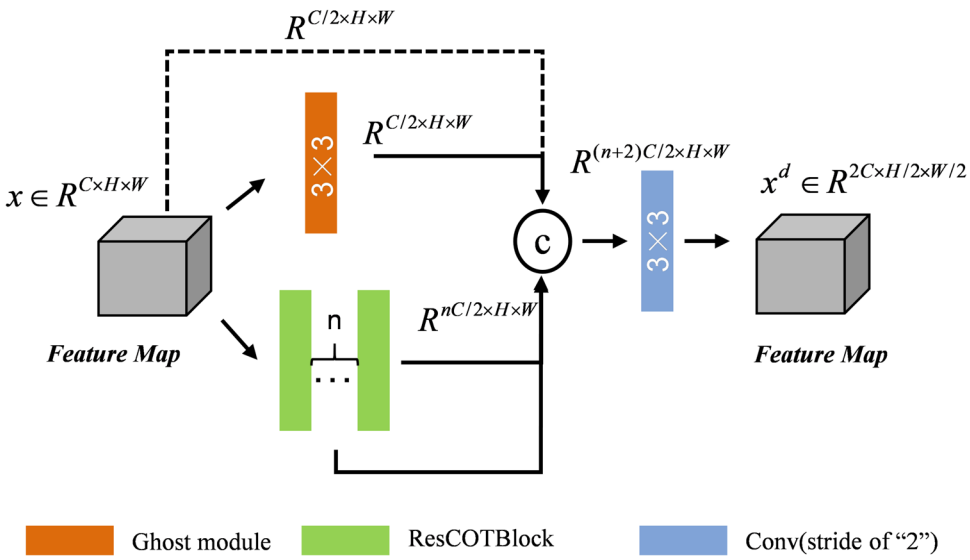
The bottleneck module is a basic module proposed to solve the problem of gradient vanishing or gradient exploding due to the deepening of the network. As the network gradually deepens, the network is prone to gradient vanishing and training difficulties. In order to solve this problem, the residual block was proposed as the core component of the Darknet53 backbone network, which brought an important breakthrough to the YOLO series. However, the residual connection can only solve the problem of information loss vertically and cannot trigger the interaction of features at different spatial positions. The contextual transformer (CoT) attention mechanism [32] can make up for this shortcoming well. Not only that, it can also explore the rich context between keywords. Therefore, we introduce the contextual attention mechanism into the residual connection to construct a better ResCoTBlock bottleneck structure.

(i) CoT attention mechanism

CoT has the advantage of mining rich contexts over traditional self-attention mechanisms, and it fully utilizes the contextual information between adjacent keywords. It integrates contextual information mining and self-attention learning into a unified architecture, promotes self-attention learning in an effective way, and enhances the representative power of the output aggregated feature map. The specific structure of CoT attention mechanism is shown in Fig. 8.

In particular, assume that there is the same input 2D feature map $X_{in} \in R^{H\times W\times C}$. Key, query, and value are defined as $K = X_{in}$, $Q = X_{in}$, $V = X_{in}W_v$ respectively. The CoT block

Fig. 6 RC2A structure diagram



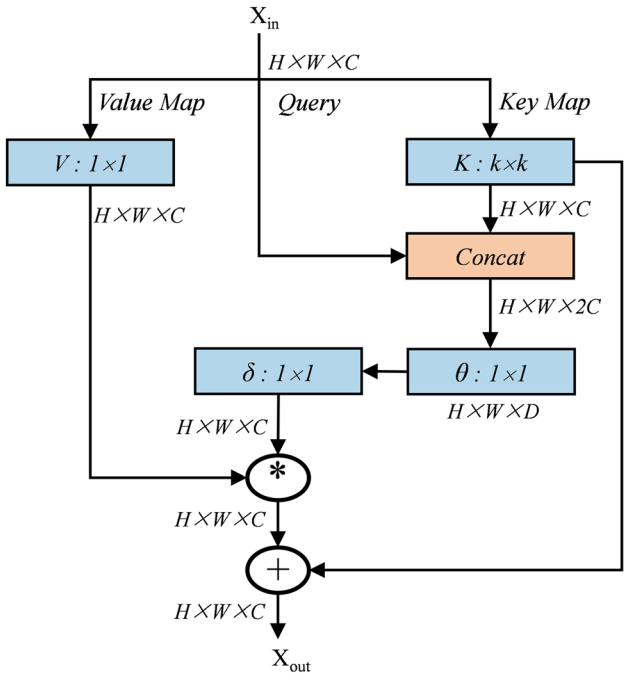


Fig. 8 Specific structure of CoT attention mechanism. + and * represent element-wise summation and local matrix multiplication respectively

first performs k×k groups of convolutions on all neighboring keywords within a k×k grid to represent each keyword as a context, instead of encoding each keyword with 1×1 convolution as in typical self-attention. The learned contextual relation $K_1 \in R^{H\times W\times C}$ reflects the static contextual information between neighboring keywords. Subsequently, the concatenation of K and query Q is conditioned on the attention matrix through two consecutive 1×1 convolutions (W_θ with ReLU activation function and W_δ without activation function):

$$A = [K_1, Q]W_\theta W_\delta \tag{5}$$

For each head, each attention matrix of A is based on feature and context information, rather than isolated query–key pairs. Next, based on the attention matrix A of context information, the attention feature map K_2 is calculated by aggregating all values V:

$$K_2 = V * A \tag{6}$$

Where K_2 is called dynamic context information. Finally, the learned dynamic context information is combined with the static context information to obtain the final feature information X_{out}:a

$$X_{out} = K_1 \oplus K_2 \tag{7}$$

Here, the final output X_{out} contains the features of each keyword and context information, and is the cascade of the aggregated feature maps of all heads.

(ii) Residual connection

Combining CoT attention with residual connections results in the ResCoTBlock bottleneck structure. By drawing on the skip connection design of ResNet, the residual block effectively alleviates the problem of gradient vanishing in deep networks and allows feature information to be directly transmitted between layers. Each residual block consists of multiple convolutional layers and skip connections. From a mathematical perspective, the residual unit can be expressed as:

$$y = x + h(x) \tag{8}$$

where x is the input feature, $h(x)$ is the operation process containing multiple components, and y is the output feature. However, using a large number of residual blocks in the network will accumulate more redundancy. In order to handle this redundant information well, this paper adds the CoT attention mechanism to the residual block to make it a new and more effective bottleneck module ResCoTBlock, as shown in Fig. 9. For ResCoTBlock, the above $h(x)$ can be expressed as:

$$h(x) = Conv(CoT(Conv(x))) \tag{9}$$

Combining CoT attention with residual blocks ensures that the original context is not lost while allowing the gradient to more centrally optimize the weight distribution of

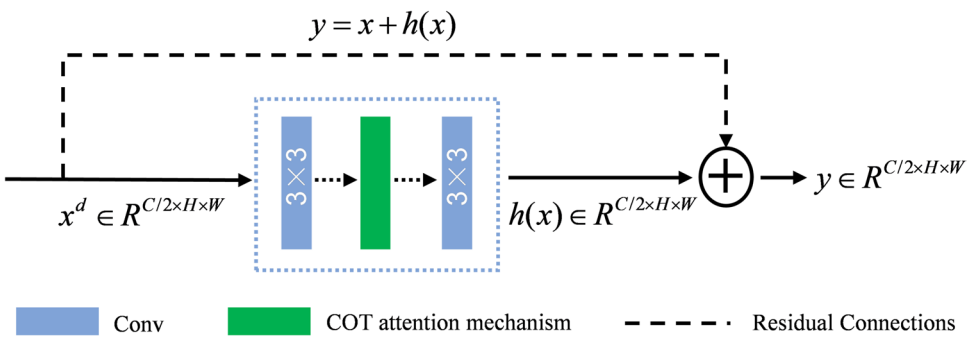


Fig. 9 Structure of ResCoTBlock

key features, thereby suppressing irrelevant and redundant information.

3.3 Design of SPPF_WD

SPPF_WD is a pyramid pooling network, which comes from the optimization of SPP. It is usually used at the end of the backbone network because it can generate multi-scale feature information to supply the neck layer of the model and increase the diversity of the neck layer input. In this paper, the main task of SPPF_WD is to provide a larger receptive field so that the model can accurately identify categories such as seat belts because seat belts often span the diagonal of the entire image, which adds challenges to large target detection and requires a larger receptive field. If we increase the receptive field in the middle layer of the backbone network, the detection effect of small targets will be reduced. In FAN-YOLO, a larger part of the terminal features of the backbone network contributes to large target detection, and has little effect on the detection of smaller targets. Therefore, we choose to optimize SPPF at the end of the backbone network, abandon the method of fixed receptive field size (such as 3, 5, 9), and adopt an incremental receptive field with a pooling operation kernel of 5, which significantly helps the detection of large targets. Our statement was proved by experiments.

4 Experimental section

4.1 Set-up

The experimental results of this paper were conducted in Pytorch 1.11.0, Python 3.8, and Support Nvidia CUDA. All experiments were conducted on Ubuntu20.04 LTS operating system, which is equipped with Intel(R) Xeon(R) Platinum 8474C 16 vCPU, 80GB memory and NVIDIA's Ada Lovelace architecture [GeForce RTX 4090 GPU] with 24GB VRAM.

All experiments in this paper follow the settings of YOLOv8 using the same head and training configuration. Other settings, such as optimizer learning rate, stochastic gradient descent (SGD) momentum, and cosine decay learning rate, are also the same as YOLOv8. In addition, mosaic and mixup are used for data enhancement, but mosaic enhancement is canceled in the last 10 rounds. The image input size for all experiments is 640×640, no pre-training is performed, and the batch size is 32. Use 70% as training data, 20% of images for verification, and the remaining 10% for testing.

4.2 Data description

The purpose of this article is for the model to determine the type of driver behavior and the location where it occurs based on the input driver distraction image. Therefore, real data is crucial to the research of the model. There are many resources in the field of driver behavior classification. This paper enhances the State Farm and YawDD [33] datasets, adds some categories and annotates them to obtain the enhanced datasets State Farm v2 and YawDD v2 datasets:

State Farm v2 Dataset: Most of the images in this dataset come from the State Farm dataset, which is publicly available in the Kaggle competition. This dataset is a new dataset obtained by annotating labels and supplementing some categories. It contains a total of 9043 images of different driver behaviors as shown in Table 1.

YawDD v2 Dataset: This dataset is mainly derived from the supplement of YawDD dataset. We have added some categories and pictures to the YawDD dataset, making it more focused on capturing the driver's facial information and body movements, judging the driver's driving status from whether the mouth is open, eyes are closed, hand position, whether the steering wheel is steady, whether the hand is holding a cup, etc. The specific content is shown in Table 2.

This paper uses the YOLO series of algorithms and other mainstream algorithms to evaluate the two datasets, with an average accuracy of up to 85% and around 90%. We believe that this dataset can serve as a solid foundation for future driver behavior detection and contribute to progress in this field.

4.3 Experimental analysis

The focus of this paper is to evaluate the size and the accuracy of the deployed model. Specifically, this paper hopes that the FAN-YOLO model can reduce the number of model

Table 1 State Farm v2 Dataset

Class	Class name	Number of images
C0	Distracted-Left	1354
C1	Distracted-Right	1525
C2	Distracted-Behind	922
C3	Distracted-Phone	1320
C4	Seatbelt	1147
C5	Smoking	1281
C6	Drinking	1068
C7	Drowsy	1373
Total	—	9043

Table 2 YawDD v2 Dataset

Class	Class name	Number of images
C0	Close-eye	899
C1	Open-eye	3066
C2	Close-mouth	2340
C3	Open-mouth	1480
C4	Hand	4059
C5	Phone	2187
C6	Cup	1234
C7	Wheel	4048
Total	—	7960

parameters and computational complexity while maintaining or even improving the existing accuracy. In addition, in order to have a more realistic comparison result, the network starts from scratch using its training data.

This paper selects N series and S series from the YOLO series network for experimental comparison as shown in Table 3. YOLOv6-S and EfficientNetv2 have too many parameters and the amount of calculation reaches 43.0G and 52.2G, which is inconsistent with our original intention of designing lightweight networks, so no experimental results are given. In the table, YOLOv8-S has the best detection effect in our experiment, but our model size is only 2.2M and the amount of calculation is only 6.2G, which is 77.6% and 73.7% lower than YOLOv8-S, and the detection speed is also 72.7% higher than YOLOv8-S. If we focus on YOLOv8-N, whose model size and scale are at the same level as our model FAN-YOLO, we can achieve a 18.5% reduction in the number of parameters and a 10.1% reduction in the amount of computation, while increasing AP_{50}^{val} and AP_{95}^{val} by 1.3% and 1.7% on the State Farm v2 dataset, and increasing AP_{50}^{val} and AP_{95}^{val} by 0.6% and 1.7% on the YawDD v2 dataset. The speed of reasoning has also been increased by about 30%. Figure 10 clearly illustrates this conclusion. In addition, compared with other lightweight detection networks, the model FAN-YOLO proposed in this article is superior to them in all aspects, especially the detection accuracy is much higher. The detection effect is shown in Fig. 11.

Tables 4 and 5 show the detection results of each category of the FAN-YOLO model on the State Farm v2 dataset and YawDD v2 dataset respectively. As can be seen from Table 4, the FAN-YOLO model has a relatively sluggish detection effect on the Seatbelt category, and the detection effect on other categories is more accurate. From Table 5, it can be seen that FAN-YOLO shows a relatively accurate detection effect when dealing with the driver's distracted facial expressions and body movements.

Figure 12 shows the confusion matrix of the FAN-YOLO model trained on the State Farm v2 dataset and the YawDD v2 dataset. Figure 13 shows the training curves of

Table 3 Detection results on the State Farm v2 dataset (1) and the YawDD v2 dataset (2)

Model	Param	FLOPs	Latency	AP^{val}(%)		AP_{50}^{val}(%)		AP_{95}^{val}(%)	
	(M)	(G)	(ms)	1	2	1	2	1	2
YOLOv5-N	2.2	5.9	2.6	86.5	93.3	85.2	96.6	62.1	73.3
YOLOv5-S	7.8	19.0	3.5	90.1	95.8	86.9	97.6	65.1	75.8
YOLOv6-N	4.2	11.6	2.9	86.5	93.2	83.3	96.7	62.1	74.3
YOLOv6-S	16.1	43.0	–	–	–	–	–	–	–
YOLOv7-tiny	6.0	13.8	3.2	86.4	92.3	83.4	96.6	57.8	69.7
YOLOv8-N	2.7	6.9	2.7	89.1	93.8	85.5	96.9	63.6	74.5
YOLOv8-S	9.8	23.6	5.5	91.9	94.9	88.5	97.6	66.8	76.6
YOLOv9-S	7.1	26.4	7.6	87.9	93.0	89.8	96.8	68.4	74.7
YOLOv10-N	2.7	8.4	2.4	82.0	93.2	81.5	96.2	61.5	73.1
YOLOv10-S	8.1	24.8	3.7	88.8	94.9	86.8	97.4	65.8	76.1
GhostNet	3.7	6.0	2.2	84.5	90.3	79.5	93.6	59.0	67.8
FasterNet	2.7	7.6	3.6	79.6	89.8	76.8	92.8	57.5	63.9
EfficientNetv1	5.5	9.4	4.8	84.9	91.2	82.5	94.8	60.9	69.0
EfficientNetv2	22.9	52.2	–	–	–	–	–	–	–
ShuffleNetv1	4.1	12.3	3.7	84.1	90.0	80.6	93.4	58.0	65.8
ShuffleNetv2	4.0	10.2	4.6	80.6	88.6	81.3	92.4	59.7	64.3
RepViT	3.1	10.1	6.8	84.2	89.3	83.7	94.4	62.7	68.1
MobileNetv2	2.4	6.7	6.1	82.7	87.1	79.9	89.7	60.0	63.3
MobileNetv3	4.2	7.5	6.4	83.2	90.3	79.9	93.3	58.6	66.3
MobileNetv4	3.1	5.8	4.1	80.9	86.5	76.6	89.7	55.5	64.4
FAN-YOLO(ours)	2.2	6.2	1.9	89.1	94.6	86.8	97.5	65.3	75.5

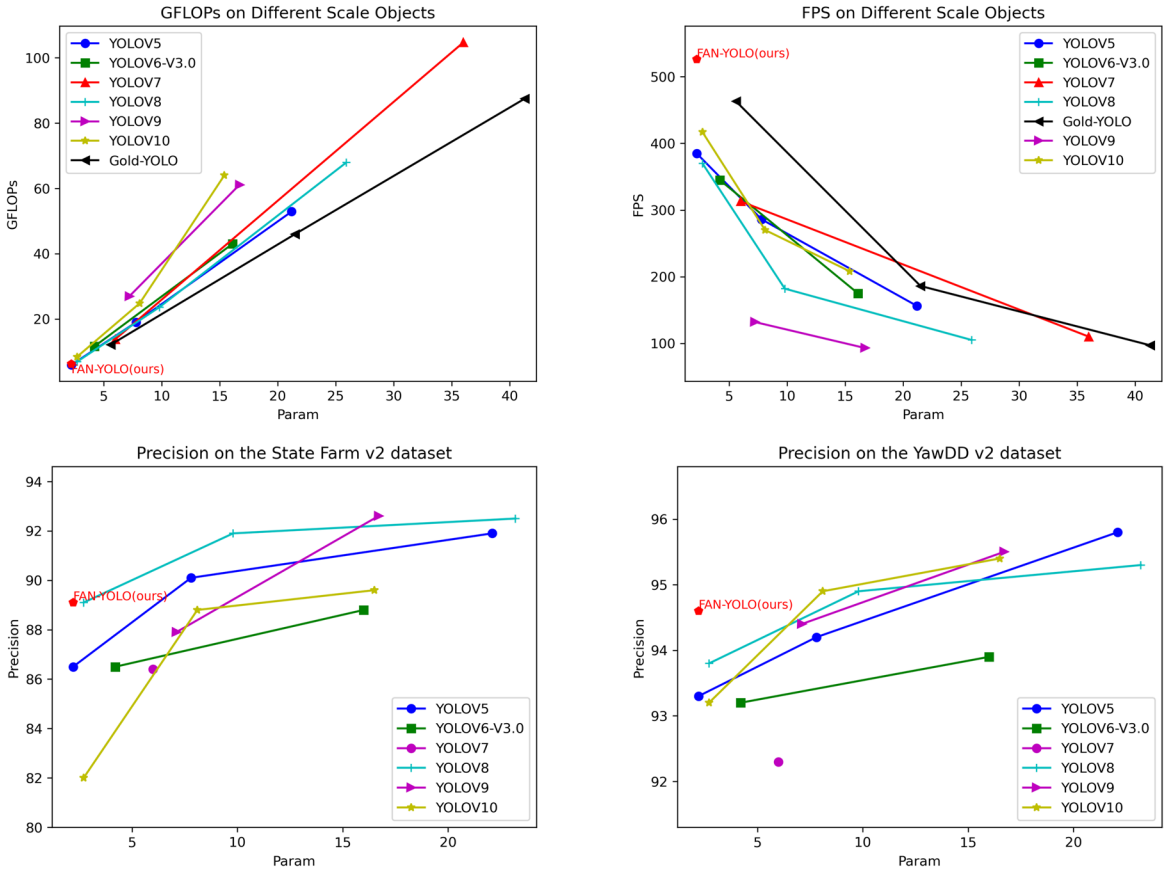
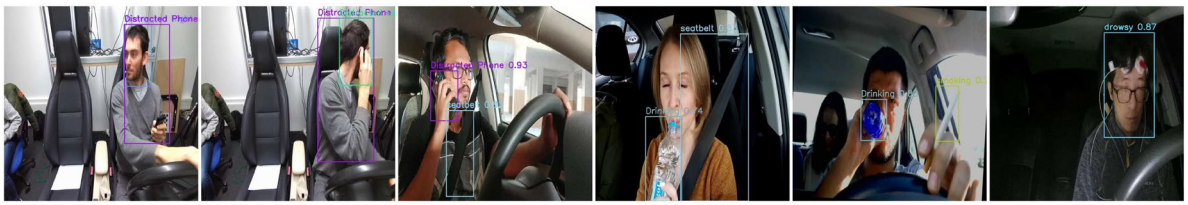
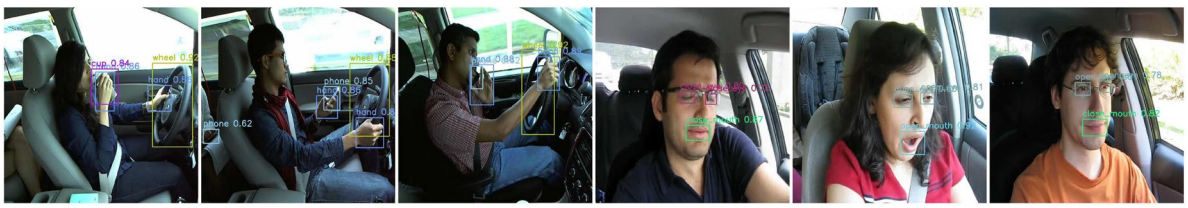


Fig. 10 Performance on real-time object detection of different sizes



(a)Detection results of 6 images in the State Farm v2 dataset.



(b)Detection results of 6 images in the YawDD v2 dataset.

Fig. 11 FAN-YOLO detection visualization on State Farm v2 dataset and YawDD v2 dataset

the FAN-YOLO model on the State Farm v2 dataset and the YawDD v2 dataset. The left image is the detection performance of AP^{val}, the middle image is the detection performance of AP^{val}_{50}, and the right image is the detection performance of AP^{val}_{95}. As can be seen from Fig. 13, the FAN-YOLO model has a faster convergence speed and maintains a high detection accuracy.

Table 4 FAN-YOLO detection results of various categories on the State Farm v2 dataset

Class	$AP^{val}(\%)$	$AP^{val}_{50}(\%)$	$AP^{val}_{95}(\%)$
Distracted-Left	97.8	99.3	85.6
Distracted-Right	99.0	98.9	87.5
Distracted-Behind	95.5	99.0	85.5
Distracted-Phone	94.2	90.9	71.8
Seatbelt	65.1	48.6	24.4
Smoking	82.2	71.6	33.1
Drinking	90.0	87.8	55.3
Drowsy	98.9	99.4	79.7

Table 5 FAN-YOLO detection results of various categories on the YawDD v2 dataset

Class	$AP^{val}(\%)$	$AP^{val}_{50}(\%)$	$AP^{val}_{95}(\%)$
Close_eye	94.9	97.3	75.5
Open_eye	94.9	98.4	67.1
Close_mouth	97.1	98.2	74.6
Open_mouth	92.1	96.4	75.8
Hand	97.4	98.9	80.4
Phone	98.4	98.9	71.5
Cup	89.7	94.9	78.0
Wheel	99.3	99.0	95.0

4.4 Experimental comparison

Since categories such as seat belts in real driving environments usually cross the diagonal of image data and are difficult to detect, in order to explore the impact of SPPF_WD on the detection of such targets, this paper conducts experiments on the State Farm v2 dataset. Through the experiments, we can know that compared with SPP, AP^{val}, AP^{val}_{50} and AP^{val}_{95} are improved by 3.18%, 2.48% and 2.43%. Compared with SPPF, SimSPPF and SPPELAN, the accuracy is also improved to a certain extent. The experimental results are shown in Table 6.

Due to the importance of SPPF_WD for seatbelt detection, we conducted a separate study on the seatbelt images in the State Farm v2 dataset. SPPF_WD is much more effective in detecting seatbelts than SPP. The detection accuracy of seatbelts using SPP is only 61.38%, while SPPF_WD reaches 64.99%, which is 3.61% higher than SPP. Not only that, the AP^{val}_{50} reaches 49.05%, which is 4%~5% higher than SPPF, SimSPPF and SPPELAN. The experimental results are shown in Table 7.

In order to explore the improvement of CoT attention on the bottleneck module, this paper experiments on the bottleneck module composed of multiple different attention mechanisms on the YawDD v2 dataset. The experiment shows that the CoT attention used in this paper has certain advantages in detection accuracy. Compared with the traditional convolution bottleneck module, CoT has increased AP^{val}, AP^{val}_{50} and AP^{val}_{95} by 1.29%, 0.54% and 1.61% respectively when the parameters remain unchanged. And compared with other attention mechanisms, there are certain improvements. The experimental results are shown in Table 8.

4.5 Ablation experiment

Ablation experiments are performed on the feature fusion modules RC2A, SPPF_WD and FAN proposed in this paper. The ablation experiments are conducted on the YawDD v2 dataset. The experimental results are shown in Table 9.

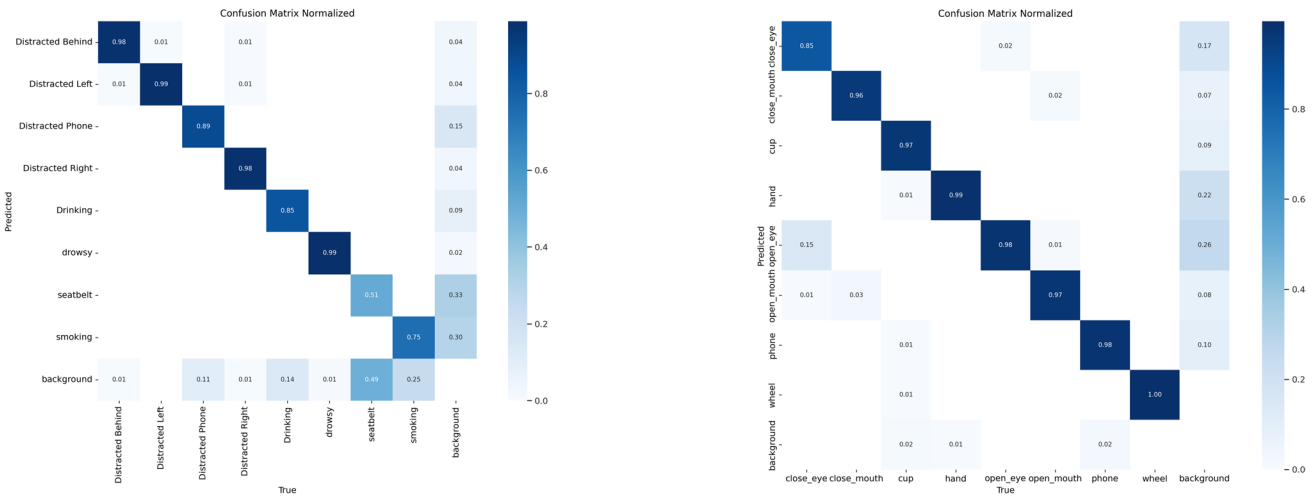
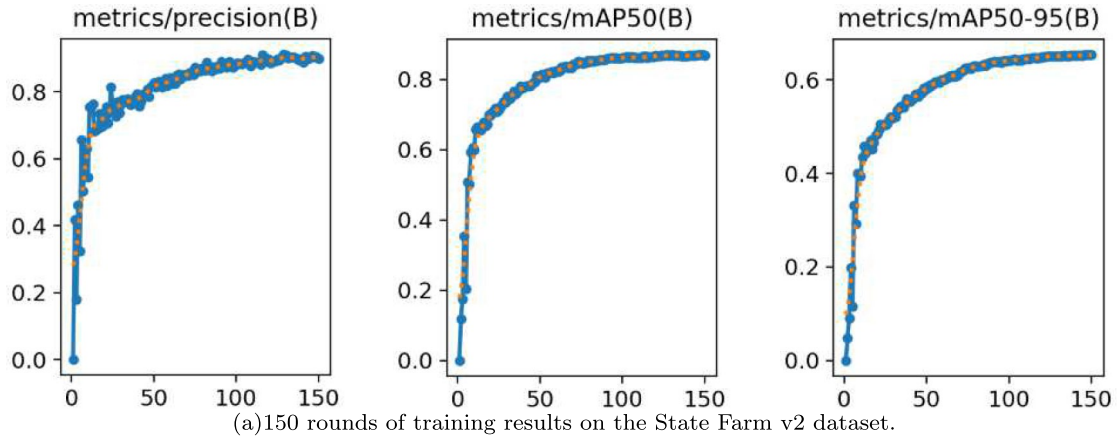
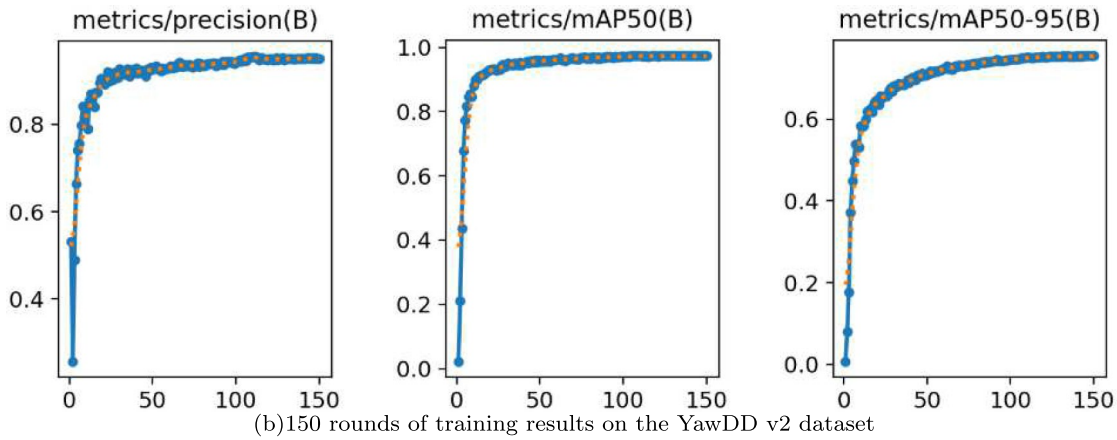


Fig. 12 Confusion matrix of FAN-YOLO on State Farm v2 dataset and YawDD v2 dataset



(a)150 rounds of training results on the State Farm v2 dataset.



(b)150 rounds of training results on the YawDD v2 dataset

Fig. 13 FAN-YOLO training results on the State Farm v2 dataset and the YawDD v2 dataset. The left side shows the training performance of AP^{val}, the middle shows the training performance of AP^{val}_{50}, and the right side shows the test performance of AP^{val}_{95}.

Table 6 Impact of SPPF_WD on the model

Model	CB type	AP^{val}(%)	AP^{val}_{50}(%)	AP^{val}_{95}(%)
FAN-YOLO	+SPP	85.92	84.29	62.88
	+SPPF	87.53	85.13	62.76
	+SimSPPF	87.84	84.78	62.87
	+SPPELAN	88.26	84.85	63.81
	+SPPF_WD	89.10	86.77	65.31

Table 7 Impact of SPPF_WD on seatbelt detection in State Farm v2 dataset

Model	CB type	AP^{val}(%)	AP^{val}_{50}(%)	AP^{val}_{95}(%)
FAN-YOLO	+SPP	61.38	42.12	20.02
	+SPPF	63.45	44.93	20.45
	+SimSPPF	63.82	44.31	20.40
	+SPPELAN	64.48	44.86	22.16
	+SPPF_WD	64.99	49.05	23.69

By comparing the experimental results of groups 1, 2, and 3, it can be seen that the FAN proposed in this paper can significantly improve the detection accuracy compared with RC2A and SPPF_WD when used alone. As shown in Fig. 14, the traditional FPN + PAN Neck layer structure cannot correctly detect all targets at the same level, but the FAN structure proposed in this paper can more accurately identify and detect all targets.

Next, by comparing the experimental groups 1, 2, 4 with those 3, 6, 7, it is easy to see the influence of RC2A and SPPF_WD on the model detection results. After adding RC2A, SPPF_WD and FAN to the model, the detection results AP^{val}_{50} and AP^{val}_{95} also reached the highest 97.48% and 75.52%.

5 Conclusion

In this paper, we annotated two datasets that can be used for driver distraction detection, and designed a new lightweight and real-time detection model FAN-YOLO based on



(a) (b) (c)

Fig. 14 GradCAM visualization of the PAN layer and the FAN layer. Figure a is the original image, Figure b is the visualization of the PAN layer, and Figure c is the visualization of the FAN layer

Table 8 Impact of different attention on the model

Model	CB type	$AP^{val}(\%)$	$AP^{val}_{50}(\%)$	$AP^{val}_{95}(\%)$
FAN-YOLO	+Conv	93.36	96.94	73.91
	+SA	94.46	97.02	74.31
	+SE	94.37	96.95	74.18
	+CBAM	94.76	96.95	73.76
	+EMA	93.26	96.78	73.90
	+SimAM	93.06	96.99	73.86
	+SK	93.65	97.27	74.54
	+CoT	94.59	97.48	75.52

this task. In FAN-YOLO, we introduced the CoT attention mechanism and designed a bottleneck module that is superior to previous ones, aiming to improve the effectiveness and efficiency of information fusion and transmission and avoid unnecessary losses. In addition, the traditional feature pyramid network and spatial pyramid pooling layer architectures are re-examined, and the FAN and SPPF_WD architectures that are more suitable for driver distraction behavior detection are developed, which significantly improves the detection ability of the model. Finally, we sincerely hope that our work can prove valuable in solving real-world problems while also bringing new ideas to researchers in this field.

Table 9 Ablation experiment of FAN-YOLO structure

Model	RC2A	SPPF_WD	FAN	$AP^{val}(\%)$	$AP^{val}_{50}(\%)$	$AP^{val}_{95}(\%)$
1	✓			93.84	96.84	74.05
2		✓		92.81	97.03	74.48
3			✓	94.47	96.96	74.54
4	✓	✓		94.03	97.05	73.89
5		✓	✓	94.44	96.95	74.35
6	✓	✓	✓	94.59	97.48	75.52

Acknowledgements We are hugely grateful to the possible anonymous reviewers for their constructive comments with respect to the original manuscript.

Author contributions Xinsu Gan: Writing-original draft, Software, Methodology, Conceptualization. Lidong Huang: Writing-review & editing, Validation, Funding acquisition. Yan Deng: Formal analysis, Data curation. Yan Yuan: Supervision, Project administration.

Funding The authors declare that no funding was received for this work.

Data availability The source code for the experiments and related research data will be made available upon reasonable request.

Declarations

Conflict of interest The authors declare no conflict of interest.

Consent for publication The authors declare that consent to publish has been obtained for any images containing potential human faces, in accordance with ethical guidelines.

References

1. World Health Organization. Regional approach to the decade of action for road safety 2021-2030." Regional approach to the decade of action for road safety 2021-2030 (2022)
2. Eraqi, Hesham M., et al.: Driver distraction identification with an ensemble of convolutional neural networks. Journal of advanced transportation **2019**(1), 4125865 (2019)
3. Berri, Rafael A., et al.: "A pattern recognition system for detecting use of mobile phones while driving." 2014 International conference on computer vision theory and applications (VISAPP). Vol. 2. IEEE (2014)
4. Seshadri, K., Juefei-Xu, F., Pal, D. K., Savvides, M., Thor, C. P.: Driver cell phone usage detection on strategic highway research program (SHRP2) face view videos. In Proceedings of the IEEE conference on computer vision and pattern recognition workshops, pp. 35-43 (2015)
5. Abouelnaga, Y., H.M. Eraqi, Moustafa, M.N.: "Real-time distracted driver posture classification. arXiv 2017." arXiv preprint arXiv:1706.09498 (2017)
6. Tran, Duy, et al.: "Real-time detection of distracted driving based on deep learning." IET Intelligent Transport Systems 12.10 :1210-1219 (2018)
7. Wu, Chenxia, et al.:"Watch-n-patch: unsupervised learning of actions and relations." IEEE transactions on pattern analysis and machine intelligence 40.2 : 467-481 (2017)
8. Baheti, B., Gajre, S., Talbar, S.: Detection of distracted driver using convolutional neural network. In Proceedings of the IEEE conference on computer vision and pattern recognition workshops, pp. 1032-1038 (2018).
9. Lu, M., Hu, Y., Lu, X.: Driver action recognition using deformable and dilated faster R-CNN with optimized region proposals. Applied Intelligence **50**(4), 1100–1111 (2020)
10. Majdi, M.S., Ram, S., Gill, J.T., Rodríguez, J.J.: Drive-net: Convolutional network for driver distraction detection. In 2018 IEEE Southwest Symposium on Image Analysis and Interpretation (SSIAI), pp. 1-4. IEEE (2018)
11. Shen, J., Liu, N., Sun, H., Li, D., Zhang, Y.: An instrument indication acquisition algorithm based on lightweight deep convolutional neural network and hybrid attention fine-grained features. IEEE Transactions on Instrumentation and Measurement **73**, 1–16 (2024)
12. Poon, Y.S., Kao, C.Y., Wang, Y.K., Hsiao, C.C., Hung, M.Y., Wang, Y.C., Fan, C.P.: Driver distracted behavior detection technology with YOLO-based deep learning networks. In 2021 IEEE International Symposium on Product Compliance Engineering-Asia (ISPCE-ASIA), pp. 01-05. IEEE. (2021)
13. Amira, Belmekki Ghizlene, Zoulikha, Mekkakia Maaza, Hector, Pomares: Driver drowsiness detection and tracking based on YOLO with Haar cascades and ERNN. International Journal of Safety and Security Engineering **11**(1), 35–42 (2021)
14. Qin, Long, et al.: ID-YOLO: Real-time salient object detection based on the driver's fixation region. IEEE Transactions on Intelligent Transportation Systems **23**(9), 15898–15908 (2022)
15. Zhou, Keke, et al.: "An Improved Distraction Behavior Detection Algorithm Based on YOLOv5." Computers, Materials Continua 81.2 (2024)
16. Debsi, Ali, et al.: "Driver distraction and fatigue detection in images using ME-YOLOv8 algorithm." IET Intelligent Transport Systems 18.10: 1910-1930 (2024)
17. Lin, T. Y., Dollár, P., Girshick, R., He, K., Hariharan, B., Belongie, S.: Feature pyramid networks for object detection. In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 2117-2125 (2017)
18. Liu, S., Qi, L., Qin, H., Shi, J., Jia, J.: Path aggregation network for instance segmentation. In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 8759-8768 (2018)
19. Kaiming, He., et al.: Spatial pyramid pooling in deep convolutional networks for visual recognition. IEEE transactions on pattern analysis and machine intelligence **37.9**, 1904–1916 (2015)
20. Redmon, J., Divvala, S., Girshick, R., Farhadi, A.: You only look once: Unified, real-time object detection. In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 779-788 (2016)
21. Redmon, J., Farhadi, A.: YOLO9000: better, faster, stronger. In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 7263-7271 (2017)
22. Redmon, Joseph, Farhadi, Ali: "Yolov3: An incremental improvement." arXiv preprint arXiv:1804.02767 (2018)
23. Bochkovskiy, Alexey, Wang, Chien-Yao, Liao, Hong-Yuan Mark: "Yolov4: Optimal speed and accuracy of object detection." arXiv preprint arXiv:2004.10934 (2020)
24. Jocher, Glenn, et al.: "ultralytics/yolov5: v6. 1-tensorrt, tensorflow edge tpu and openvino export and inference." Zenodo (2022)
25. Li, Chuyi, et al.: "YOLOv6: A single-stage object detection framework for industrial applications." arXiv preprint arXiv:2209.02976 (2022)
26. Wang, C.Y., Bochkovskiy, A., Liao, H.Y.M.: YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 7464-7475 (2023)
27. Varghese, R., Sambath, M.: Yolov8: A novel object detection algorithm with enhanced performance and robustness. In 2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS), pp. 1-6. IEEE (2024)
28. Wang, C. Y., Yeh, I. H., Mark Liao, H. Y.: Yolov9: Learning what you want to learn using programmable gradient information. In European conference on computer vision (pp. 1-21). Cham: Springer Nature Switzerland (2024)
29. Wang, Ao, et al.: "Yolov10: Real-time end-to-end object detection." Advances in Neural Information Processing Systems 37: 107984-108011 (2025)
30. Ding, X., Zhang, X., Ma, N., Han, J., Ding, G., Sun, J.: Repvgg: Making vgg-style convnets great again. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition, pp. 13733-13742 (2021)
31. Han, K., Wang, Y., Tian, Q., Guo, J., Xu, C., Xu, C.: Ghostnet: More features from cheap operations. In Proceedings of the IEEE/CVF

conference on computer vision and pattern recognition, pp. 1580-1589 (2020)
32. Li, Y., Yao, T., Pan, Y., Mei, T.: Contextual transformer networks for visual recognition. IEEE transactions on pattern analysis and machine intelligence **45**(2), 1489–1500 (2022)
33. Abtahi, S., Omidyeganeh, M., Shirmohammadi, S., Hariri, B.: YawDD: A yawning detection dataset. In Proceedings of the 5th ACM multimedia systems conference, pp. 24-28 (2014)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.